

МОДУЛЬ 1

АДМИНИСТРИРОВАНИЕ LINUX

ПРОГРАММИРОВАНИЕ НА SHELL

ПЕРЕМЕННЫЕ SHELL

Переменные окружения

- ☐ Определяются на уровне пользовательского контекста
- ☐ Передаются дочерним процессам

Позиционные параметры

- ☐ Передаются через параметры скрипта при вызове
- ☐ Действительны только для текущего скрипта
- ☐ Могут включать служебные параметры (\$@, \$#, \$\$)

Локальные переменные

- ☐ Определяются в блоках кода скрипта
- ☐ Имеют различные области видимости и жизни

ОПРЕДЕЛЕНИЕ ПЕРЕМЕННЫХ

- Имена переменных чувствительны к регистру

```
a=10
```

```
echo "value: $a"           #value: 10
```

```
A=15
```

```
echo "value: $a"           #value: 10
```

```
echo "value: $A"           #value: 15
```

- Определение переменных происходит при их инициализации

```
a=10      ;      b="str_value"
```

- Переменные shell не типизированы

```
a=10
```

```
echo "Number value: $a" #Number value: 10
```

```
a="string"
```

```
echo "String value: $a" #String value: string
```

ОБРАЩЕНИЕ К ПЕРЕМЕННЫМ

- Обращение к переменным происходит через операцию разыменования \$

```
MyVar=123
```

```
echo "Variable value = $MyVar"
```

- Обращение к переменным через \${...}

```
a=${MyVar}
```

- Изменение значений переменных

- Используя команду let

```
let "MyVar=$MyVar+1"
```

```
let MyVar=$MyVar+1
```

```
let MyVar=MyVar+1
```

ОБРАЩЕНИЕ К ПЕРЕМЕННЫМ

- Используя команду `((...))` - аналог `let`

`( (MyVar=$MyVar+1) )`

`( (MyVar=MyVar+1) )`

`a=$( (MyVar+1) )`

МАССИВЫ

- Формат определения массива:

Array = (Value1 Value2 Value3 . . . ValueN);

Array = (One Two Three Four)

- Формат присвоения значения элементу массива:

Array[Index] = 10;

Array[1]=\$A; Array[Iter]=\$A;

- Формат обращения к элементу массива:

A = \${Array[Index]}

A = \${Array[1]}; A = \${Array[Iter]};

КОСВЕННЫЕ ССЫЛКИ НА ПЕРЕМЕННЫЕ

```
#!/bin/bash
```

```
# Косвенные ссылки на переменные
```

```
a=letter_of_alphabet
```

```
letter_of_alphabet=z echo
```

```
# Прямое обращение к переменной
```

```
echo "a = $a"
```

```
# Косвенное обращение к переменной
```

```
eval a=\$$a
```

```
echo "А теперь a = $a" echo
```

ПОЗИЦИОННЫЕ ПАРАМЕТРЫ

- Доступны через обращения `${Num}`, где Num – это порядковый номер параметра:

```
echo "Started ${0} with key ${1}"
```

- В параметре `${0}` всегда находится полное имя запущенного скрипта

```
echo "Is started now: ${0}"
```

- Для каждого скрипта определяются дополнительные позиционные параметры

ДОПОЛНИТЕЛЬНЫЕ ПОЗИЦИОННЫЕ ПАРАМЕТРЫ

\$# - количество параметров, переданных скрипту в командной строке

\$* - все параметры, переданные скрипту в виде одной строки

\$@ - все параметры переданные скрипту в виде массива строк

\$! - PID последнего процесса, запущенного в фоновом режиме

\$\$ - PID самого процесса, в котором выполняется сценарий

\$? - код завершения последней команды

```
#ВЫЗОВ скрипта      MyScript  -p  First_Parameter  
  
echo    "Num params:  $0"  #!/bin/ MyScript  
  
echo    "Num params:  $#"  #3  
  
echo    "Params:  $*"      # -p
```

First_Parameter

ЗАДАНИЕ

1. Написать скрипт, который подсчитает количество слов в строке без использования команды `wc`

ПРОВЕРКА УСЛОВИЙ

`[ ... ]` | `test` | `[[...]]` - зарезервированные слова `bash` для проверки условий;

- возвращают 1 в случае истинности условия и 0 в случае , если условие ложно

`[ 5 > 3 ]`

пробелы

`echo "[ 5 > 3 ]"`

`#1`

`echo "[ 5 < 3 ]"`

`#0`

`echo "test 5 > 3"`

`#1`

`echo "test 5 < 3"`

`#0`

ОПЕРАТОРЫ УСЛОВИЙ

Сравнение целых чисел

-eq – равны

```
if [ $MyVar -eq 0 ]
```

-gt - больше (аналог <)

-ge - больше или равно (аналог <=)

-ne - не равны

-lt - меньше (аналог >)

-le - меньше или равно (аналог >=)

Операторы проверки файлов

-e – файл существует

```
if [-e My.txt ]
```

-s - ненулевой размер файла

-r - доступен для чтения

-x - доступен для исполнения

-f - обычный файл

```
if [-f My.txt ]
```

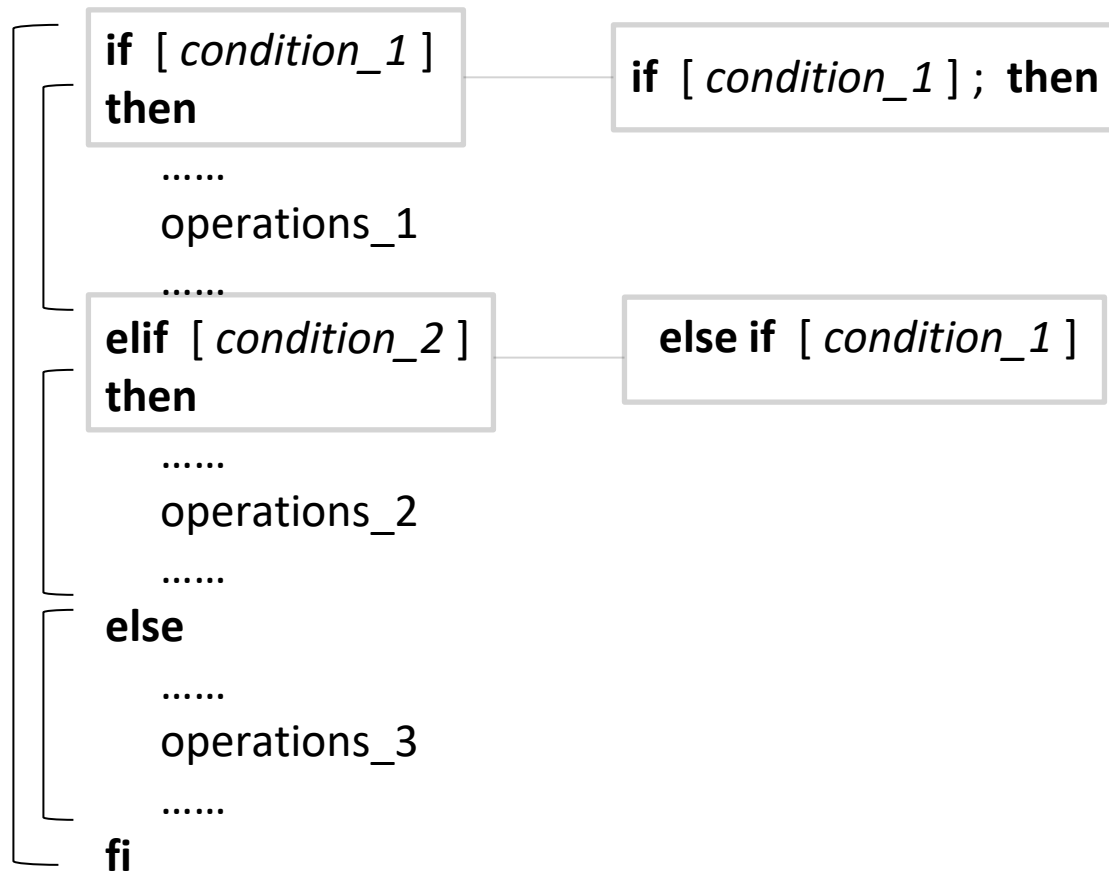
-d - файл является каталогом

-w - доступен для записи

КОНСТРУКЦИЯ IF-ELSE

- Операции в теле блока выполняются, если истинно условие *condition*
- В качестве условия могут использоваться коды возврата команд

Синтаксис:



КОНСТРУКЦИЯ FOR

- Итерация по списку значений:

```
for  iterator  in list_of_values
do
    .....
    operations_1
    .....
done
```

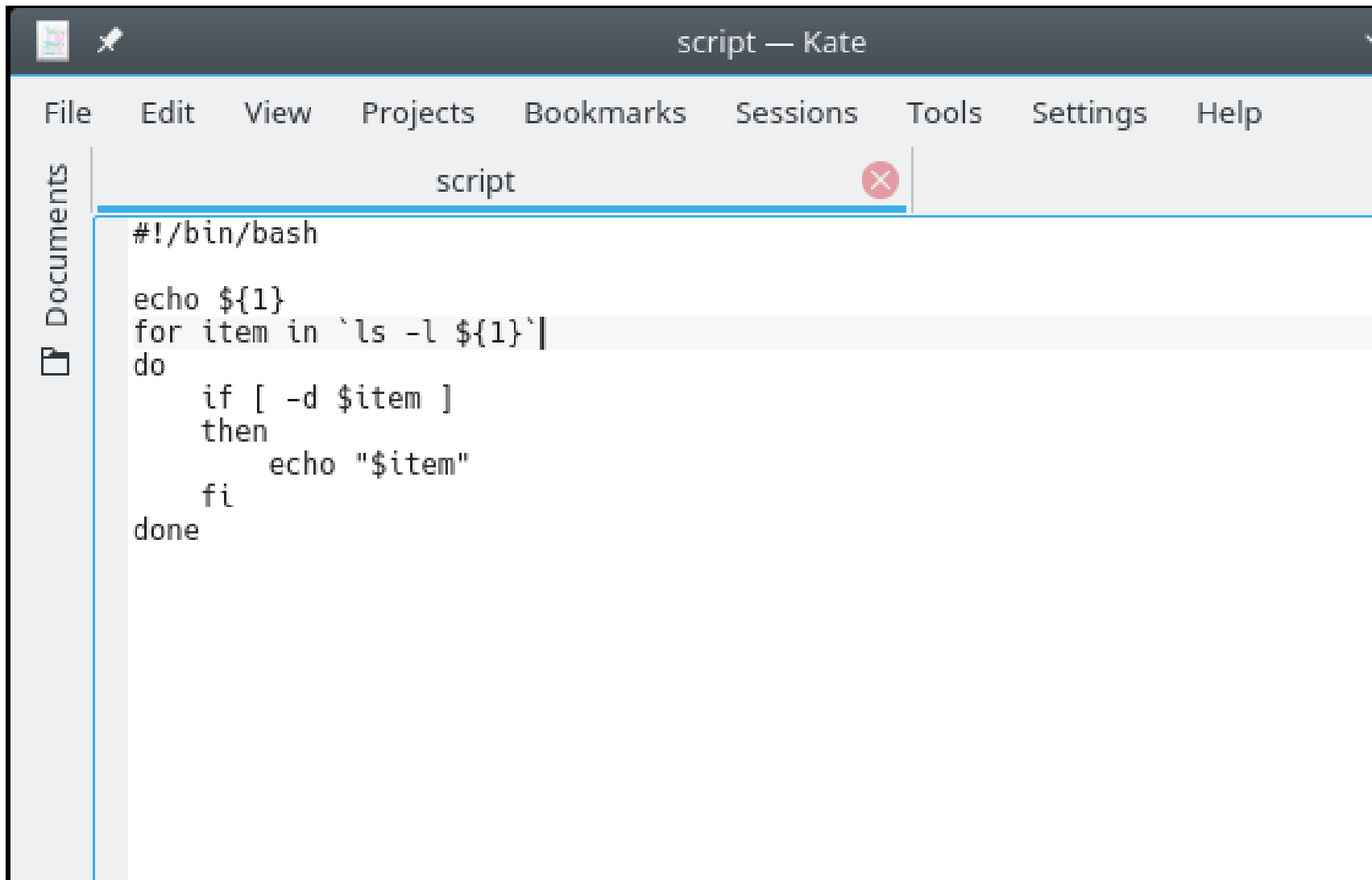
Пример:

```
for  Var  in "Val1"  "Val2"
do
    echo "Vals:  $Var"
done
```

ЗАДАНИЕ

1. Написать скрипт, который выведет на экран список имен вложенных директорий. Целевая директория передается в качестве параметра
1. Написать скрипт, который конвертирует uuid, передаваемый в параметрах, в имя устройства

РЕШЕНИЕ № 1

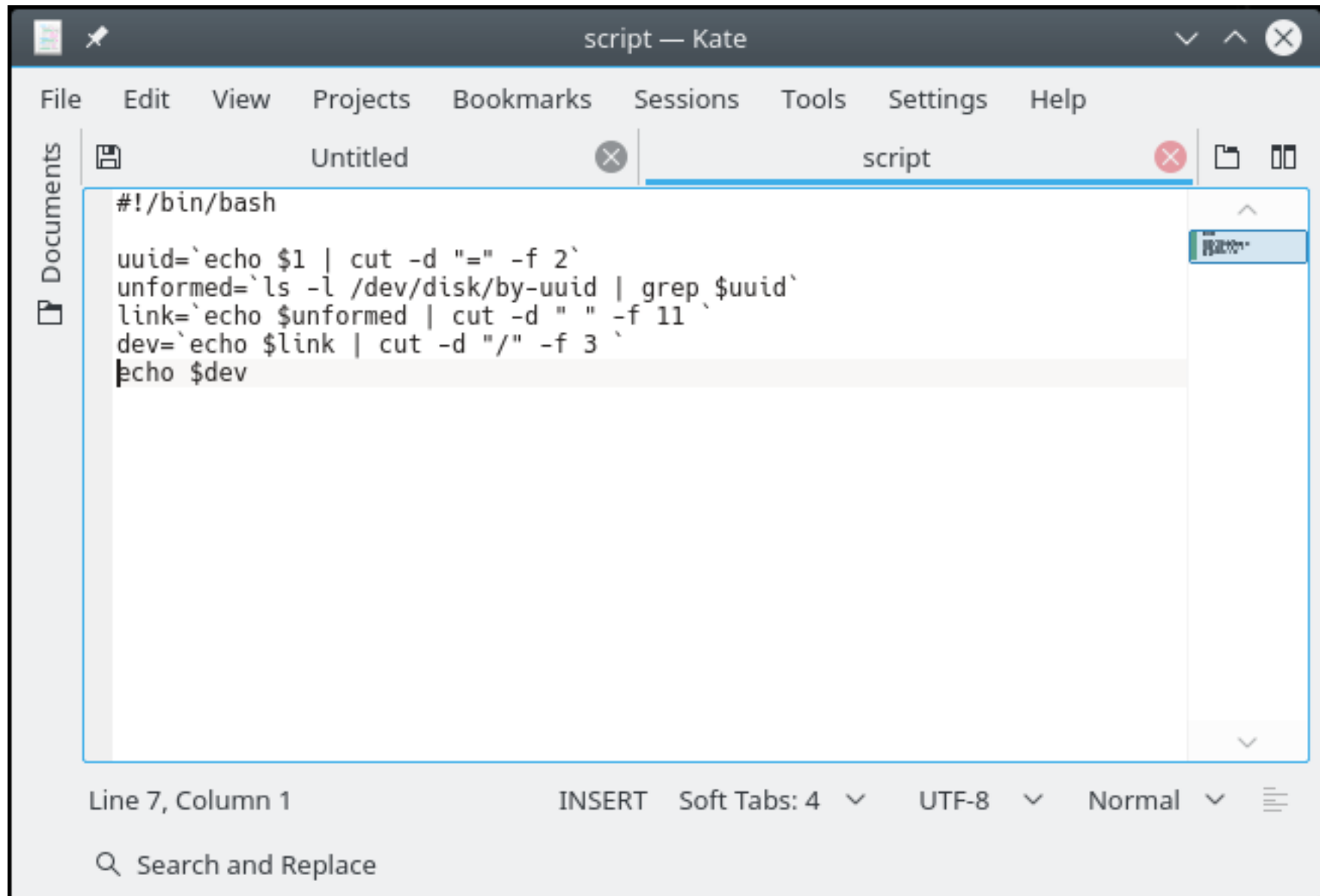


The image shows a screenshot of the Kate text editor interface. The title bar at the top reads "script — Kate". Below the title bar is a menu bar with the following items: File, Edit, View, Projects, Bookmarks, Sessions, Tools, Settings, and Help. On the left side, there is a sidebar with a "Documents" icon and the word "Documents". The main editing area contains a file named "script", which is highlighted with a blue border and a red close button. The script's content is as follows:

```
#!/bin/bash

echo ${1}
for item in `ls -l ${1}`
do
    if [ -d $item ]
    then
        echo "$item"
    fi
done
```


РЕШЕНИЕ № 2



The image shows a screenshot of the Kate text editor window. The title bar reads "script — Kate". The menu bar includes "File", "Edit", "View", "Projects", "Bookmarks", "Sessions", "Tools", "Settings", and "Help". The tab bar shows two tabs: "Untitled" and "script". The "script" tab is active and contains the following shell script code:

```
#!/bin/bash

uuid=`echo $1 | cut -d "=" -f 2`
unformed=`ls -l /dev/disk/by-uuid | grep $uuid`
link=`echo $unformed | cut -d " " -f 11`
dev=`echo $link | cut -d "/" -f 3`
echo $dev
```

The status bar at the bottom indicates "Line 7, Column 1", "INSERT" mode, "Soft Tabs: 4", "UTF-8" encoding, and "Normal" font style. A search bar labeled "Search and Replace" is visible in the bottom left corner.

КОНСТРУКЦИЯ FOR

- Итерация по счетчику:

```
for  (( iterator; condition;  
changes))  
do  
      .....  
      operations_1  
      .....  
done
```

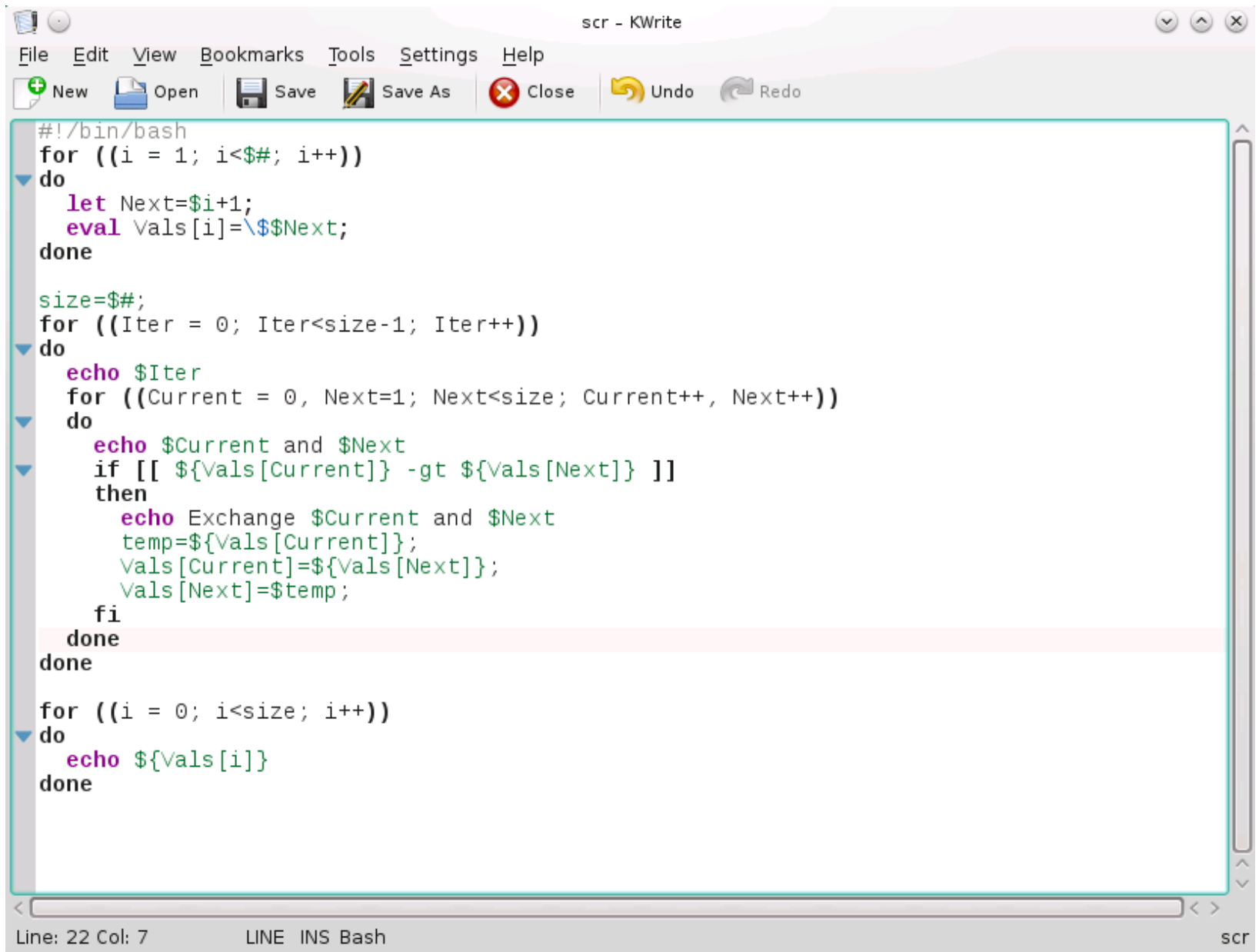
Пример:

```
for  (( a=1; a<4; a++))  
do  
      echo "Vals: $a"  
done
```

ЗАДАНИЕ

1. Написать скрипт, который сортирует переданные ему числовые параметры в порядке их возрастания

РЕШЕНИЕ



The screenshot shows a KWrite text editor window titled "scr - KWrite". The menu bar includes File, Edit, View, Bookmarks, Tools, Settings, and Help. The toolbar contains icons for New, Open, Save, Save As, Close, Undo, and Redo. The editor contains a Bash script that sorts an array of numbers. The script uses nested loops and conditional statements to compare and swap elements. The status bar at the bottom indicates "Line: 22 Col: 7" and "LINE INS Bash".

```
#!/bin/bash
for ((i = 1; i<${#}; i++))
do
    let Next=i+1;
    eval Vals[i]=\${Vals[Next];
done

size=${#};
for ((Iter = 0; Iter<size-1; Iter++))
do
    echo $Iter
    for ((Current = 0, Next=1; Next<size; Current++, Next++))
    do
        echo $Current and $Next
        if [[ ${Vals[Current]} -gt ${Vals[Next]} ]]
        then
            echo Exchange $Current and $Next
            temp=${Vals[Current]};
            Vals[Current]=${Vals[Next]};
            Vals[Next]=$temp;
        fi
    done
done

for ((i = 0; i<size; i++))
do
    echo ${Vals[i]}
done
```

Line: 22 Col: 7 LINE INS Bash scr

КОНСТРУКЦИЯ WHILE

- Операции в теле цикла выполняются пока истинно условие *condition*

```
while [ condition ]  
do  
    .....  
    operations  
    .....  
done
```

```
while (( condition ))  
do  
    .....  
    operations  
    .....  
done
```

Примеры:

```
while [ a -le 3 ]  
do  
    let a+=1  
    ...  
done
```

```
while (( a <= 3 ))  
do  
    (( a++ ))  
    ...  
done
```

```
while :  
do  
    (( a++ ))  
    ...  
done
```

КОНСТРУКЦИЯ UNTIL

- Операции в теле цикла выполняются пока ложно условие *condition*

```
until [ condition ]  
do  
    .....  
    operations  
    .....  
done
```

```
until (( condition ))  
do  
    .....  
    operations  
    .....  
done
```

Примеры:

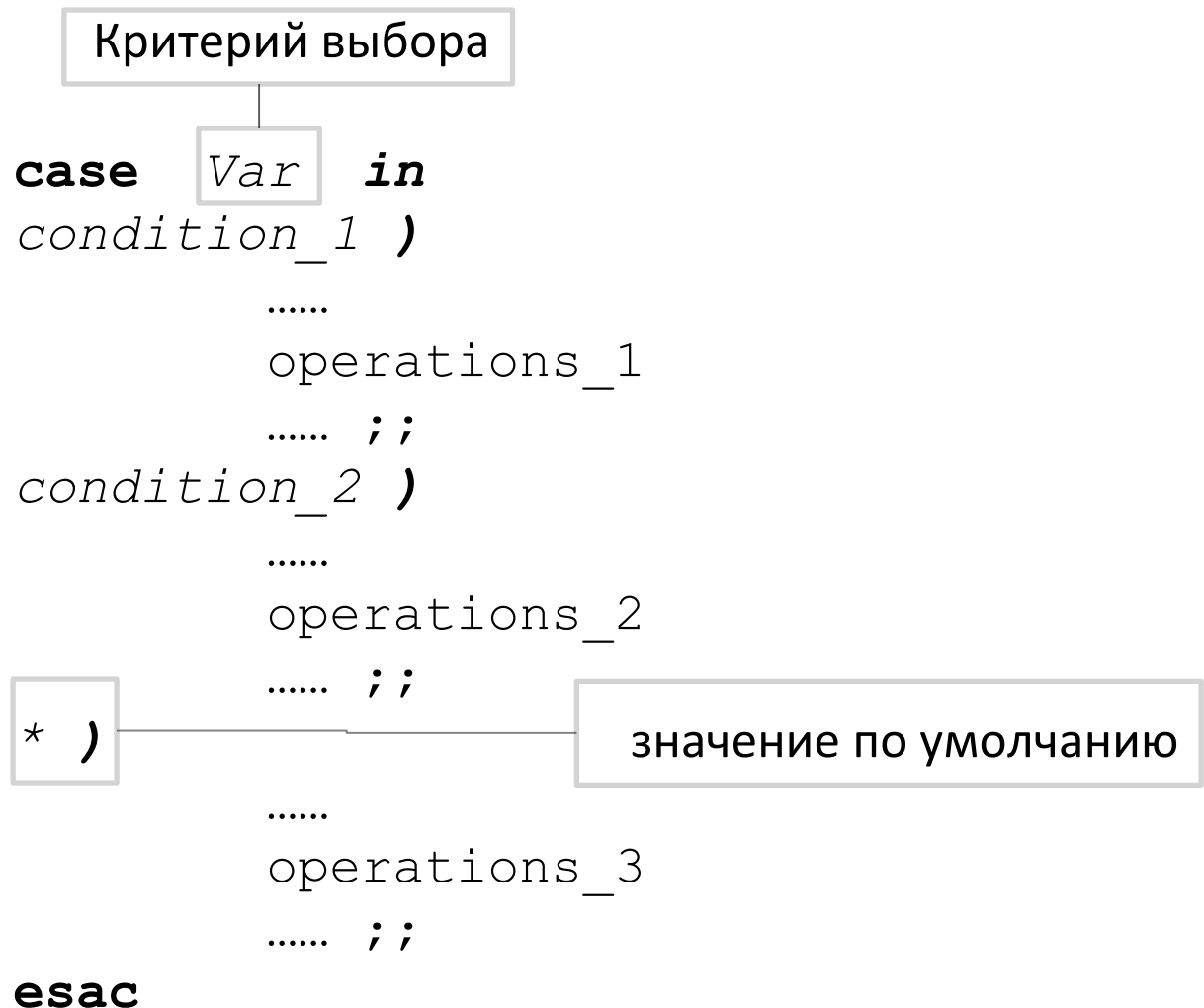
```
until [ a -gt 3 ]  
do  
    let a--=1  
    ...  
done
```

```
until (( a>3 ))  
do  
    (( a-- ))  
    ...  
done
```

```
until :  
do  
    (( a++ ))  
    ...  
done
```

КОНСТРУКЦИЯ CASE

- Используется для выбора между несколькими условными операторами:



КОНСТРУКЦИЯ CASE

Примеры:

```
case $V in  
1 )  
    echo "$V"  
    echo "First" ;;  
  
2)  
    echo "$V"  
    echo "Second" ;;  
  
* )  
    echo "$V"  
    echo "Third" ;;  
esac
```

```
case $V in  
$Cond1 )  
    echo "$V"  
    echo "First" ;;  
  
$Cond2)  
    echo "$V"  
    echo "First" ;;  
  
* )  
    echo "$V"  
    echo "First" ;;  
esac
```


ПЕРЕНАПРАВЛЕНИЕ БЛОКОВ КОДА

```
#перенаправление ввода из файла InputFile.txt;  
#действительно для всего блока кода
```

```
while      :  
    do  
        read  var1  
        echo  $var1  
    done < InputFile.txt
```

```
#перенаправление вывода в файл InputFile.txt;  
#действительно для всего блока кода
```

```
while      :  
    do  
        read  var1  
        echo  $var1  
    done >> InputFile.txt
```

ФУНКЦИИ

Функция – это именованный блок кода, вызов которого возможен по имени с передачей ему параметров.

- Определение через ключевое слово ***function***

```
function    Func_Name  
{  
    .....  
    operations  
    .....  
}
```

- Укороченный формат определения

```
Func_Name (  
{  
    .....  
    operations  
    .....  
}
```

ВЛОЖЕННЫЕ ФУНКЦИИ

- Определение функции внутри тела другой функции:

```
OutFunc ()  
{  
    InFunc ()  
    {  
        .....  
    }  
    .....  
    InFunc  
}
```

- Определение функции внутри конструкций:

```
If [ "$USER"="root" ]  
then  
    InFunc ()  
    {  
        .....  
    }  
    .....  
    InFunc  
fi
```



Функция должна быть определена до ее вызова. В противном случае скрипт завершится с ошибкой.

ПАРАМЕТРЫ ФУНКЦИИ

- Осуществляется аналогично получению значений позиционных параметров через структуру `$n`, где `n` – это индекс параметра
- `$0` содержит имя запущенного скрипта
- `$@`, `$*` - содержат списки аргументов функции
- `$#` - содержит количество параметров, переданных функции

Пример:

```
Test_func()  
{  
    echo "In the script $0: Number params - $#"  
}  
  
Test_func 1 2 3
```

ВОЗВРАТ ЗНАЧЕНИЙ ИЗ ФУНКЦИИ

- Возврат через дополнительный позиционный параметр \$?

```
Test_func()  
{  
    return $({1}+1)  
}  
  
/* * * * *  
  
Test_func 10  
VarB=$?
```



Можно передавать только числовой параметр, значение которого не превышает 255

ВОЗВРАТ ЗНАЧЕНИЙ ИЗ ФУНКЦИИ

- Передача через поток вывода

```
Test_func()  
{  
    echo $( (${1}+1) )  
}  
  
/* * * * *  
  
VarB=$(Test_func 10)
```

ЗАДАНИЕ

1. Написать рекурсивную функцию вычисления факториала передаваемого ей числа
1. Добавить в рекурсивную функцию вывод на экран промежуточного результата

РЕШЕНИЕ № 1

```
#!/bin/bash

function factorial
{
    if [ $1 -eq 1 ] then
        echo 1
    else
        temp=$(( $1 - 1 ))
        result=$(factorial $temp)
        echo=$(( $result * $1 ))
    fi
}

echo $(factorial $value)
```


ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА

Перенаправление потоков ввода/вывода — это передача входных и выходных байтовых потоков между файлам и процессами.

Для каждой оболочки всегда открыты 3 файла:

- ☐ 0 – stdin – дескриптор стандартного ввода
- ☐ 1 – stdout – дескриптор стандартного вывода
- ☐ 2 – stderr – дескриптор стандартной ошибки



Для открываемых файлов номера дескрипторов начинаются с 10.

Дескрипторы 3-9 зарезервированы для операций с дескрипторами стандартного ввода-вывода.

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

Перенаправление вывода (>):

- Перенаправление вывода команды в файл с перезаписью

Формат:

```
<Command> > <File>
```

Пример:

```
ls -l > 1.txt
```

- Очистка содержимого

Формат:

```
: > <File>  
> <File>
```

Пример:

```
: > 1.txt
```

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

Перенаправление вывода с добавлением(>>):

- Перенаправление вывода команды в файл с добавлением

Формат:

```
<Command> >> <File>
```

Пример:

```
ls -l >> 1.txt
```

- Перенаправление stdout и stderr

Формат:

```
&> j  
&> <File>
```

Пример:

```
&> 1.txt
```

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

Пример:

```
# Перенаправление вывода (stdout) в файл "filename"
1>filename

# Перенаправление вывода (stdout) в файл "filename", файл
#открывается в режиме добавления
1>>filename

# Перенаправление stderr в файл "filename"
2>filename

# Перенаправление stderr в файл "filename", файл
#открывается в режиме добавления
2>>filename

# Перенаправление stdout и stderr в файл "filename"
&>filename
```

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

Перенаправление потоков между собой:

- Перенаправление ввода от *i* в *j*

Формат:

```
i >& j
```

Пример:

```
1 >& 2
```

- Перенаправление вывода из файла

Формат:

```
<Command> < j  
<Command> < <File>
```

Пример:

```
grep .d < 1.txt
```

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)



Перенаправления могут соединяться между собой в одной

конструкции `<Command> < <Input_File> > <Output_File>`

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

Открытие и закрытие дескрипторов:

- ☐ Открытие дескриптора

Формат:

```
i <> <File>
```

Пример:

```
4 <> 1.txt
```

- ☐ Закрытие входного дескриптора i

Формат:

```
i <n-
```

Пример:

```
1 <n-
```

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

- Заккрытие выходного дескриптора i

Формат:

i >n-

Пример:

2 >n-

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ С ПОМОЩЬЮ EXEC

Используется для постоянного перенаправления потоков ввода/вывода внутри командной оболочки

```
exec <Forward_Instructions>
```

Пример:

```
exec 5 <> outFile.txt  
exec 4 >& 1  
exec 1 >& 5
```

ПЕРЕНАПРАВЛЕНИЕ ПОТОКОВ ВВОДА/ВЫВОДА (2)

```
exec 4 >& 1          //сохранение stdout в 4-м дескрипторе
exec 1 > outFile     //перенаправление всего вывода оболочки

. . . .

exec 1 >& 4           //восстановление stdout из 4-го дескр-ра
```

РАБОТА С АWK